Профилирование памяти: dotnet-dump и dotMemory

C# Developer Advanced

● REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?

Дмитрий Гурьянов

Руководитель группы .NET @ SKL Group

- Разрабатываю на .NET с 2015
- Специализация –бизнес приложения на платформе Microsoft

@guryanovda

mail@dmgur.ru

Маршрут вебинара

- Знакомство
- dotnet-counters
- dotnet-dump
- dotnet-gcdump
- dotMemory

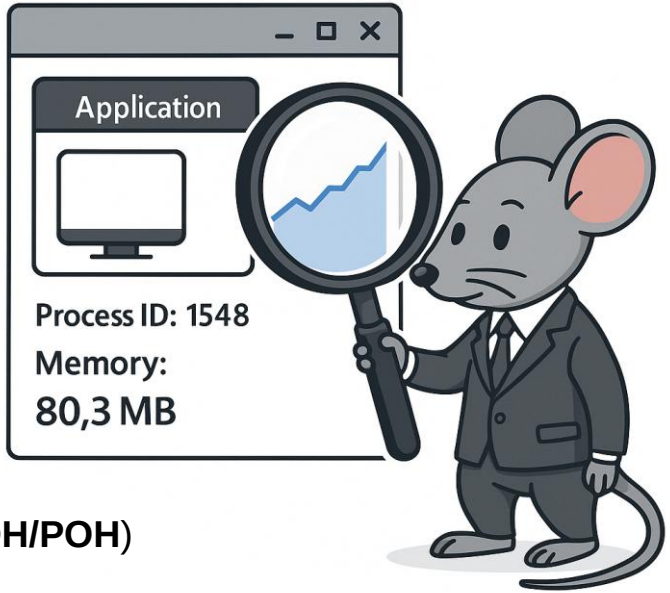
Цели вебинара

К концу занятия вы сможете

1. создавать и анализировать дампы памяти для диагностики проблем
2. находить утечки памяти, анализируя пути к корням объектов (GC roots)
3. выявлять источники избыточных аллокаций, которые приводят к частым сборкам мусора

Профилирование памяти

Симптомы что у приложения проблемы с памятью



Что видно снаружи:

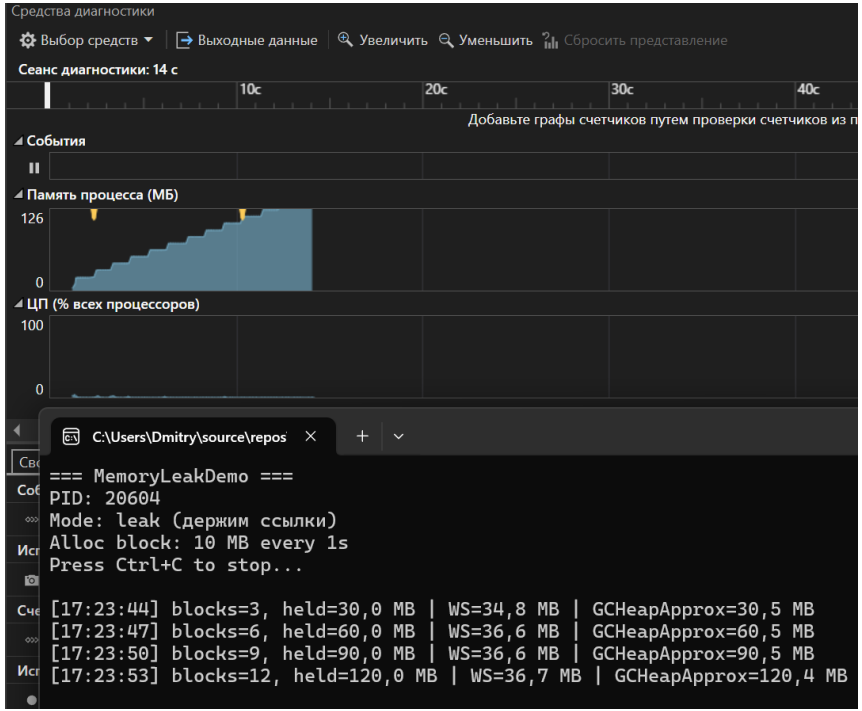
1. Рост **Private Bytes / RSS**
2. Частые **OOM / OOMKilled** в контейнере
3. Паузы/фризы
4. Скачки скорости обработки
5. Ошибки 500 под нагрузкой

Внутренние сигналы:

1. Устойчивый рост **GC Heap Size** (особенно **LOH/POH**)
2. Высокий **Allocation Rate**, рост **Gen2 GC** и длительных пауз
3. Рост **Pinned objects**, **Finalization queue**, **GC Handles**

Демо: Рост потребления памяти

1. Каждую секунду выделяется новый массив в 10 МБ и сохраняется в списке
2. Ссылки не освобождаются — сборщик мусора не может вернуть память
3. В диспетчере задач / отладчике видно стабильный рост используемой памяти



Какие бывают проблемы с памятью в .NET

- **Утечка ссылок (retention leak)** — объект удерживается ссылкой и не может быть собран GC.

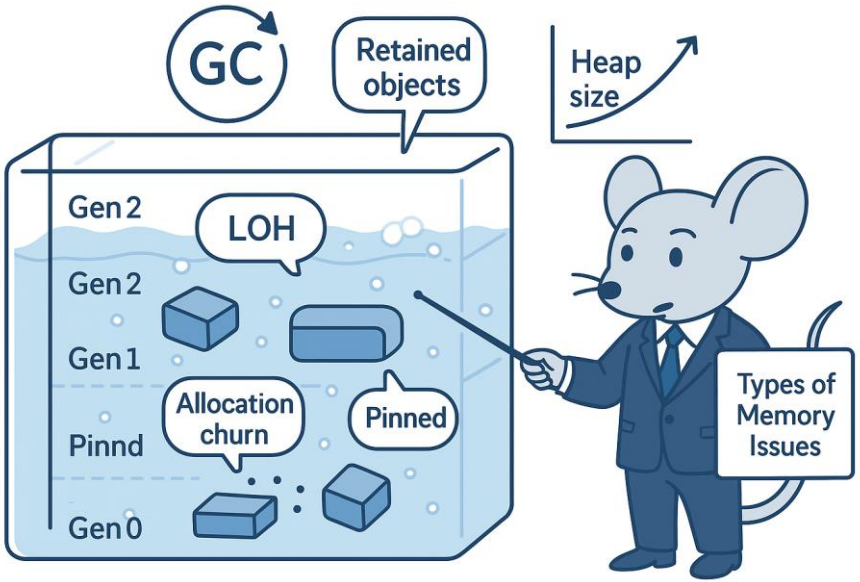
- **Чрезмерные аллокации (allocation churn)** — высокий темп аллокаций.

- **LOH-проблемы** — крупные объекты (≥ ~85 КБ) ведут к росту/фрагментации Large Object Heap.

- **Закреплённые объекты (pinned)** — закрепление мешает компактингу, усиливает фрагментацию.

- **Финализаторы/IDisposable** — забыли закрыть внешние ресурсы.

- **Неуправляемые утечки (unmanaged)**



dotnet-counters

dotnet-counters: Зачем нужен и что показывает

- Инструмент реального времени для наблюдения за .NET-процессом
- Использует EventPipe API CLR, не требует отладчика
- Позволяет мониторить счётчики GC, JIT, ThreadPool, Exceptions, ASP.NET и System.Runtime
- Работает на Windows, Linux, macOS
- Уровень детализации — обзорный, без паузы процесса
- Подходит для ранней диагностики проблем с памятью и производительностью

```
PowerShell
Press p to pause, r to resume, q to quit.
    Status: Running

Name                                                                    Current Value
[System.Runtime]
    % Time in GC since last GC (%)                                              0
    Allocation Rate (B / 1 sec)                                            10,493,984
    CPU Usage (%)                                                            0.26
    Exception Count (Count / 1 sec)                                             0
    GC Committed Bytes (MB)                                                 452.669
    GC Fragmentation (%)                                                        0
    GC Heap Size (MB)                                                       629.652
    Gen 0 GC Budget (MB)                                                        8
    Gen 0 GC Count (Count / 1 sec)                                              0
    Gen 0 Size (B)                                                            584
    Gen 1 GC Count (Count / 1 sec)                                              0
    Gen 1 Size (B)                                                          1,160
    Gen 2 GC Count (Count / 1 sec)                                              0
    Gen 2 Size (B)                                                        341,016
    IL Bytes Jitted (B)                                                    72,200
    LOH Size (B)                                                        4.5089e+08
    Monitor Lock Contention Count (Count / 1 sec)                               0
    Number of Active Timers                                                     1
    Number of Assemblies Loaded                                                13
    Number of Methods Jitted                                                  537
    POH (Pinned Object Heap) Size (B)                                      32,712
    ThreadPool Completed Work Item Count (Count / 1 sec)                        1
    ThreadPool Queue Length                                                     0
    ThreadPool Thread Count                                                     2
    Time paused by GC (ms / 1 sec)                                              0
    Time spent in JIT (ms / 1 sec)                                         13.592
    Working Set (MB)                                                       38.867

PS C:\Users\Dmitry>
```

dotnet-counters: Основные команды

Просмотр списка активных процессов:

```
dotnet-counters ps
```

Мониторинг выбранного процесса в реальном времени:

```
dotnet-counters monitor --process-id <PID>
```

Отслеживание конкретных счётчиков:

```
dotnet-counters monitor --process-id <PID> --counters System.Runtime[gc-heap-size,loh-size,alloc-rate,gen-2-gc-count]
```

Запись данных в файл для последующего анализа:

```
dotnet-counters collect --process-id <PID> --output counters.json
```

Завершение мониторинга — **Ctrl+C**.

dotnet-counters: Куда смотреть?

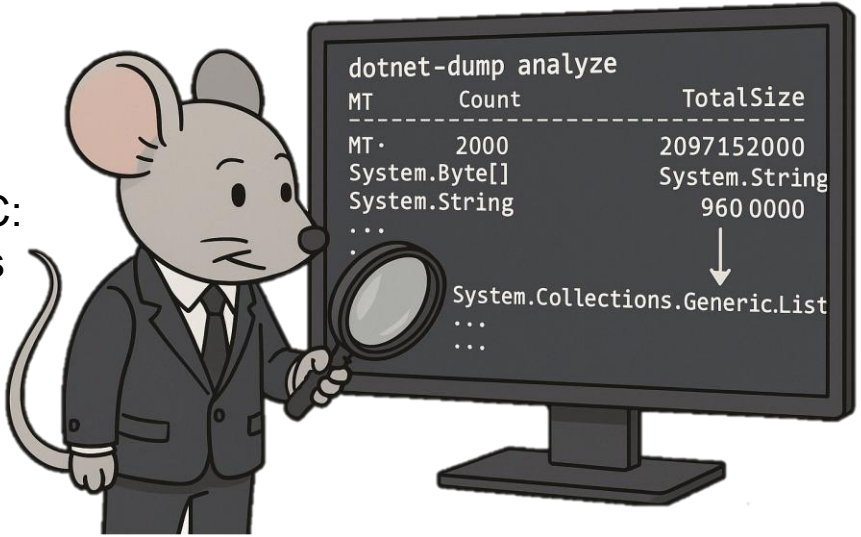
```
PowerShell

Press p to pause, r to resume, q to quit.
    Status: Running

Name                                                                    Current Value
[System.Runtime]
    % Time in GC since last GC (%)                                                  0
    Allocation Rate (B / 1 sec)                                            10,493,984
    CPU Usage (%)                                                                0.26
    Exception Count (Count / 1 sec)                                                 0
    GC Committed Bytes (MB)                                                   452.669
    GC Fragmentation (%)                                                            0
    GC Heap Size (MB)                                                         629.652
    Gen 0 GC Budget (MB)                                                            8
    Gen 0 GC Count (Count / 1 sec)                                                  0
    Gen 0 Size (B)                                                                584
    Gen 1 GC Count (Count / 1 sec)                                                  0
    Gen 1 Size (B)                                                              1,160
    Gen 2 GC Count (Count / 1 sec)                                                  0
    Gen 2 Size (B)                                                            341,016
    IL Bytes Jitted (B)                                                        72,200
    LOH Size (B)                                                            4.5089e+08
    Monitor Lock Contention Count (Count / 1 sec)                                   0
    Number of Active Timers                                                         1
    Number of Assemblies Loaded                                                    13
    Number of Methods Jitted                                                      537
    POH (Pinned Object Heap) Size (B)                                          32,712
    ThreadPool Completed Work Item Count (Count / 1 sec)                            1
    ThreadPool Queue Length                                                         0
    ThreadPool Thread Count                                                         2
    Time paused by GC (ms / 1 sec)                                                  0
    Time spent in JIT (ms / 1 sec)                                             13.592
    Working Set (MB)                                                           38.867

PS C:\Users\Dmitry>
```

LIVE

dotnet-dump

dotnet-dump: Зачем нужен и что показывает

- Кроссплатформенный сбор **полного дампа процесса** (.NET) без нативного отладчика
- Режимы: **collect** (снять дамп) и **analyze** (интерактивный разбор)
- Покрывает управляемые структуры CLR/GC: **потоки, стеки, куча, объекты, типы, roots**
- Ограничение: **не** нативный отладчик



dotnet-dump: Основные команды

Снять дамп работающего процесса:

```
dotnet-dump collect --process-id <PID> --type full
```

Открыть дамп для анализа:

```
dotnet-dump analyze core_<PID>.dmp
```

Основные команды в режиме анализа:

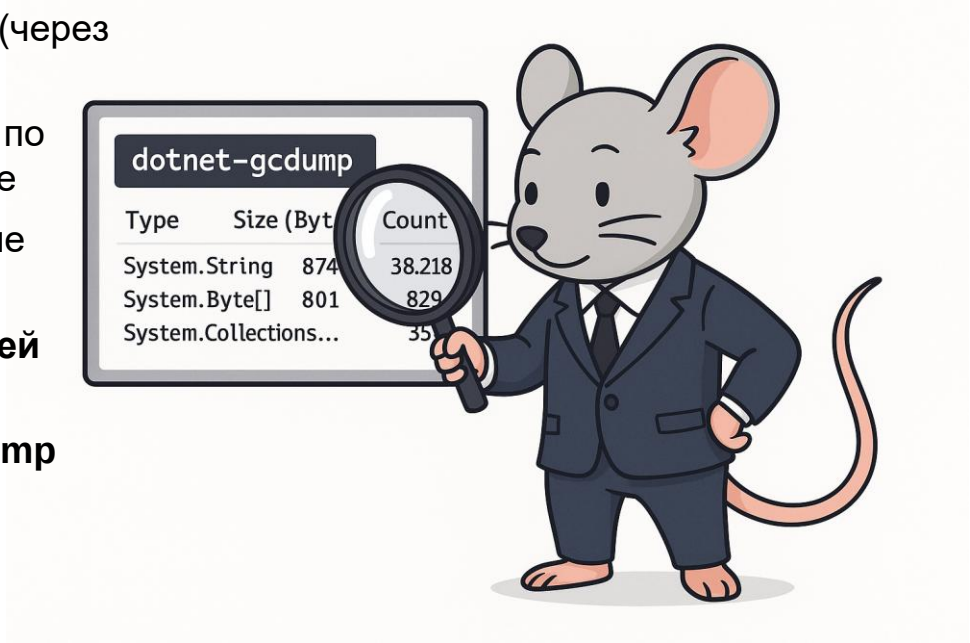
- **help** — список доступных команд
- **threads** — список потоков
- **clrstack** — стек вызовов управляемого кода
- **dumpheap -stat** — статистика по типам объектов
- **dumpobj <адрес>** — просмотр содержимого объекта
- **gcroot <адрес>** — путь удержания объекта (GC root)
- **eeheap -gc** — структура управляемой кучи (Gen0, Gen1, Gen2, LOH, POH)

LIVE

dotnet-gcdump

dotnet-gcdump: Зачем нужен и что показывает

- Лёгкий **GC-снимок** без остановки процесса (через EventPipe)
- Даёт агрегированную картину кучи: объёмы по типам, количество объектов, доминирование
- Подходит для **продакшна**: низкие накладные расходы, кратковременное подключение
- Не заменяет полный дамп: **нет стеков, полей объектов, локалов**
- Формат .gcdump открывается в **dotnet-gcdump report**, **Visual Studio**, **dotMemory**



dotnet-gcdump: Основные команды

Просмотр списка активных процессов:

```
dotnet-counters ps
```

Снять GC-снимок без остановки процесса:

```
dotnet-gcdump collect --process-id <PID> --output heap.gcdump
```

Просмотреть результаты:

```
dotnet-gcdump report heap.gcdump
```

dotnet-gcdump: Куда смотреть?

```
PS C:\Users\Dmitry> dotnet-gcdump report heap.gcdump
    210,072,206  GC Heap bytes
          4,129  GC Heap objects

   Object Bytes     Count  Type
     10,485,784        20  System.Byte[] (Bytes > 10M)  [System.Private.CoreLib.dll]
         30,534         1  System.String (Bytes > 10K)  [System.Private.CoreLib.dll]
         28,920         1  EventMetadata[] (Bytes > 10K)  [System.Private.CoreLib.dll]
         16,344         1  System.Object[] (Bytes > 10K)  [System.Private.CoreLib.dll]
          8,184         2  System.Object[] (Bytes > 1K)  [System.Private.CoreLib.dll]
          4,024         1  LogEntry[] (Bytes > 1K)  [System.Private.CoreLib.dll]
          3,608         1  System.Byte[] (Bytes > 1K)  [System.Private.CoreLib.dll]
          3,120         1  System.Int32[] (Bytes > 1K)  [System.Private.CoreLib.dll]
          2,424         1  System.String[] (Bytes > 1K)  [System.Private.CoreLib.dll]
          1,976         1  System.Reflection.RuntimeMethodInfo[] (Bytes > 1K)  [System.Private.CoreLib.dll]
            536         3  System.Char[]  [System.Private.CoreLib.dll]
            536         2  Slot<System.Object>[]  [System.Private.CoreLib.dll]
            512         1  System.Threading.PortableThreadPool  [System.Private.CoreLib.dll]
            456         3  System.Globalization.CultureData  [System.Private.CoreLib.dll]
            456         1  ThreadLocalArray<System.Char>[]  [System.Private.CoreLib.dll]
            400         1  System.Diagnostics.Tracing.RuntimeEventSource  [System.Private.CoreLib.dll]
            400         1  System.Globalization.DateTimeFormatInfo  [System.Private.CoreLib.dll]
```

LIVE

JetBrains dotMemory

dotMemory: Зачем нужен и что показывает

dump_20251031_183627.dmp – JetBrains dotMemory

Home | Analysis #1

Process Dump
#0 CLR 8.0.20

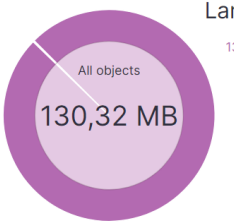
Snapshot #1
All objects

3,4 K objects
130,32 MB total size

Inspections | Types | Dominators | Similar Retention | Instances | Call Tree | Back Traces | Generations

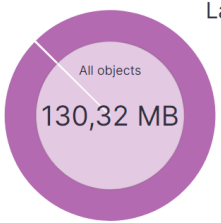
This view shows an overview of snapshot data and results of automatic memory inspections

Largest Size



130,00 MB	Byte[]
84,2 KB	String
64,3 KB	RuntimeParameterInfo
33,4 KB	Object[]
28,2 KB	EventSource+EventMetadata[]

Largest Retained Size



130,11 MB	9×Object[]
202,9 KB	NativeRuntimeEventSource
1,2 KB	Process
1,0 KB	ThreadPoolWorkQueue
832 B	ConcurrentQueue<Object>

String duplicates

A large number of strings with the same value is inefficient from the point of memory usage

21,2 KB wasted by 601 objects

	Wasted	Count	Length
"ClrInstanceID"	5,7 KB	122	13
"ModuleID"	684 B	19	8
"MethodID"	684 B	19	8
"Count"	640 B	21	5
"AppDomainID"	484 B	12	11

Sparse arrays

Partially filled arrays are inefficient from the point of memory usage

130,04 MB wasted by 20 objects

	Wasted	Fill rate	Length
↳Byte[]	10,00 MB	0,0%	10 485 760
↳Byte[]	10,00 MB	0,0%	10 485 760
↳Byte[]	10,00 MB	0,0%	10 485 760
↳Byte[]	10,00 MB	0,0%	10 485 760
↳Byte[]	10,00 MB	0,0%	10 485 760

Finalizable objects

49 objects of 19 type(s) found

Queued	Finalized	Total

Pre-calculating snapshot #1...

dotMemory: Куда смотреть?

dump_20251031_183627.dmp – JetBrains dotMemory

Home | Analysis #1

Process Dump
#0 CLR 8.0.20

Snapshot #1
All objects

3,4 K objects
130,32 MB total size

Group by generation
Large Object Heap

3 objects
30,00 MB total size

Inspections | Types | Dominators | Similar Retention | Instances | Call Tree | Back Traces | Generations

Filter

Types	Objects	Bytes	Minimum retain
Byte[] System	17	136 319 472	136 319 472
String System	1 139	86 240	86 240
RuntimeParameterInfo System.Reflection	686	65 856	98 184
Object[] System	12	34 208	136 429 518
EventSource+EventMetadata[] System.Diagnostics.Tracing	1	28 920	207 144
RuntimeMethodInfo System.Reflection	247	25 688	40 368
ParameterInfo[] System.Reflection	292	17 872	19 216
Signature System	162	12 960	22 456
RuntimeType[] System	167	9 760	10 000
String[] System	46	5 520	11 828
OpCode System.Reflection.Emit	226	5 424	5 424
PortableThreadPool+HillClimbing+LogEntry[] System.Threading	1	4 024	4 024
Int32[] System	11	3 580	3 580
RuntimeMethodInfo[] System.Reflection	4	2 072	16 368
RuntimeType System	48	1 920	1 920
CultureData System.Globalization	3	1 368	6 054
Char[] System	3	1 100	1 100
ConcurrentQueueSegment+Slot<Object>[] System.Collections.Concurrent	2	1 072	1 072
TimerQueue System.Threading	12	1 056	1 056
RuntimeType+RuntimeTypeCache System	5	800	19 314
Func<Object, PropertyValue> System	10	640	640
ScalarTypeInfo System.Diagnostics.Tracing	10	640	970
NumberFormatInfo System.Globalization	2	624	872
ThreadInfo System.Diagnostics	11	616	616

dotMemory: Куда смотреть?

dump_20251031_183627.dmp – JetBrains dotMemory

Home | Analysis #1

Process Dump
#0 CLR 8.0.20

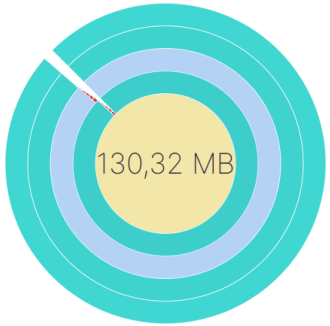
Snapshot #1
All objects
3,4 K objects
130,32 MB total size

Group by generation
Large Object Heap
3 objects
30,00 MB total size

Inspections | Types | Dominators | Similar Retention | Instances | Call Tree | Back Traces | Generations

Hide Chart | Filter

Whole Object Set



Tree | Path | Collapse all

Type	Retained byte:	Retained obj
Whole Object Set	130,32 MB	3,4 K
Object[] ×9 System	130,11 MB	801
List<Byte[]> ×1 System.Collect	130,00 MB	15
Smaller objects	78,1 KB	777
Smaller objects	214,6 KB	2,6 K

dotMemory: Куда смотреть?

dump_20251031_183627.dmp – JetBrains dotMemory

Home | Analysis #1

Process Dump
#0 CLR 8.0.20

Snapshot #1
All objects

3,4 K objects
130,32 MB total size

Group by generation
Large Object Heap

3 objects
30,00 MB total size

Inspections | Types | Dominators | Similar Retention | Instances | Call Tree | Back Traces | Generations

Filter

Type	Objects	Bytes
Byte[] System	13	136 315 192
RuntimeParameterInfo System.Reflectior	531	50 976
Object[] System	9	33 624
String System	18	32 390
EventSource+EventMetadata[] System.D	1	28 920
String System	531	26 006
RuntimeMethodInfo System.Reflection	140	14 560
RuntimeParameterInfo System.Reflectior	141	13 536
RuntimeMethodInfo System.Reflection	103	10 712
ParameterInfo[] System.Reflection	141	8 760
String System	141	8 738
ParameterInfo[] System.Reflection	140	8 688
Signature System	100	8 000
RuntimeType[] System	100	7 144
String System	141	6 322
OpCode System.Reflection.Emit	226	5 424
PortableThreadPool+HillClimbing+LogEnt	1	4 024
Byte[] System	1	3 608
Signature System	41	3 280
Int32[] System	1	3 120
String[] System	9	3 024
String System	14	2 524
String System	71	2 208
RuntimeMethodInfo[] System.Reflection	1	1 976

RuntimeParameterInfo System.Reflection
[]
ParameterInfo[]
[].Parameters
EventSource+EventMetadata[]
m_eventData
NativeRuntimeEventSource

F-Reachable Queue
m_eventSource
EventSource+OverrideEventProvider
F-Reachable Queue

dotMemory: Куда смотреть?

dump_20251031_183627.dmp – JetBrains dotMemory

Home | Analysis #1

Process Dump
#0 CLR 8.0.20

Snapshot #1
All objects

3,4 K objects
130,32 MB total size

Group by generation
Large Object Heap

3 objects
30,00 MB total size

Inspections | Types | Dominators | Similar Retention | Instances | Call Tree | Back Traces | Generations

Map	Total bytes	Utilization	Objects	Fragmentati
Pinned Object Heap				
	32 712	100%	3	0%
Large Object Heap				
	31 457 448	100%	3	0%
	31 457 448	100%	3	0%
	31 457 448	100%	3	0%
	31 457 448	100%	3	0%
	10 485 816	100%	1	0%
Generation 2				
	37 752	35%	456	84%
Generation 0				
	791 256	36%	2 903	94%

Вопросы

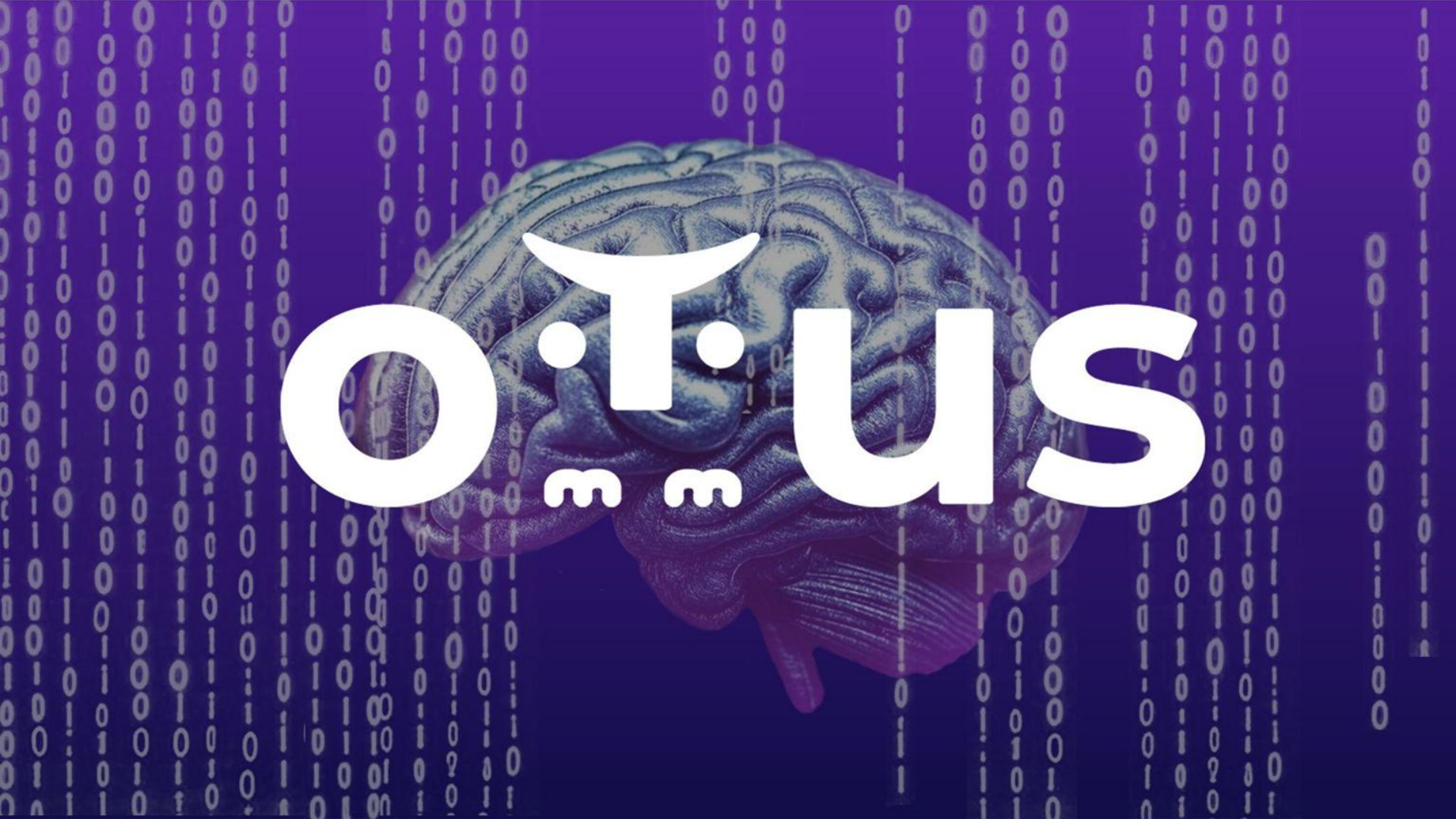
+ если есть вопросы

– если вопросов нет

Практика

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️


otus